

CONCLUSIONS I PROPOSTES DE FUTUR IAM AMB RBAC

1. RESUM

S'ha dissenyat i implementat un Simulador IAM amb RBAC, un sistema de ciberseguretat que gestiona quins usuaris poden accedir a quins recursos dins d'una organització, basant-se en rols i permisos estructurats de forma jeràrquica.

El sistema resol un problema real i crític en ciberseguretat empresarial: la gestió d'accessos manual és lenta, propensa a errors i no escala. Amb RBAC, els permisos s'assignen a rols, i els rols hereten permisos dels seus pares de forma recursiva.

2. Objectius assolits

POO amb jerarquia i polimorfisme: Classe abstracta Entitat amb subclasses polimòrfiques Usuario, Rol i Recurso. Cada subclasse implementa describir() i tipo() de forma independent.

Múltiples estructures de dades simultànies: dict per a accés $O(1)$, set per a permisos sense duplicats i arbre de punters per a l'herència de rols.

Recursivitat real i justificada: tiene_permiso() i obtener_todos_permisos() utilitzen recursió per recórrer la jerarquia de rols. Inclou protecció contra cicles amb el paràmetre 'visitados'.

Ciberseguretat aplicada: El sistema controla accés a recursos sensibles per nivell (baix/mig/alt/crític), registra tots els accessos i denegacions en un log d'auditoria immutable.

Tests unitaris complets: 26 tests unitaris que cobreixen entitats, herència de rols, verificació d'accés, polimorfisme i casos límit

Benchmark empíric: Mesura de temps reals que validen l'anàlisi teòrica i confirmen el comportament $O(1)$ per a operacions bàsiques i $O(r \cdot h \cdot p)$ per a verificació.

3. Reflexió sobre les decisions de disseny

Per què dict i no llista per a usuaris/rols/recursos? L'accés per nom és l'operació més freqüent. Un dict proporciona $O(1)$ enfront de $O(n)$ d'una llista. En un sistema amb milers d'usuaris, la diferència és enorme.

Per què set per a permisos i no llista? Els permisos no han de repetir-se. Un set garanteix la unicitat sense necessitat de comprovar duplicats manualment, i la cerca és $O(1)$ enfront de $O(p)$ d'una llista.

Per què recursivitat per a l'herència? La jerarquia de rols és un problema naturalment recursiu: per saber si el rol té un permís, es comprova el propi rol i, si no, s'aplica la mateixa pregunta al rol pare. La solució iterativa requeriria una pila manual que obfusca la lògica.

Per què el log com a llista? El log d'auditoria és seqüencial per naturalesa. Afegir al final és $O(1)$ amortitzat, i la majoria de consultes llegeixen tot el log ($O(n)$ inevitable). Una llista és l'estructura més natural i eficient.

4. Propostes de futur

El sistema actual és un simulador funcional i correcte. Per convertir-lo en un producte de producció caldria:

- I. **Autenticació amb contrasenya + hash:** Afegir un camp `password_hash` als usuaris i un endpoint d'autenticació que retorni un token JWT vàlid.
- II. **Herència múltiple de rols:** Permetre que un rol tingui múltiples pares, convertint l'arbre en un DAG (Directed Acyclic Graph) i usant BFS per recórrer l'herència.

- III. **Persistència a base de dades:** Serialitzar l'estat a SQLite o PostgreSQL per garantir durabilitat entre reinicis.
- IV. **Interfície web de gestió:** Panell d'administració web per gestionar usuaris, rols i recursos sense necessitat de línia de comandes.
- V. **Caché de permisos efectius:** Caché invalidada en modificar rols/permisos, que reduiria `verificar_acceso()` de $O(r \cdot h \cdot p)$ a $O(1)$ per a lectures repetides.
- VI. **Polítiques temporals:** Permisos amb data d'expiració (p.ex. accés temporal per a un auditor extern), inspirat en sistemes ABAC (Attribute-Based Access Control).

5. Conclusió final

Aquest projecte ens ha demostrat com les estructures de dades i els algoritmes fonamentals no són conceptes teòrics aïllats sinó la base sobre la qual es construeixen sistemes de seguretat reals. La diferència entre una gestió d'accessos manual ($O(n)$ per operació) i un RBAC ben dissenyat ($O(1)$ per operació bàsica) pot significar la diferència entre un sistema que aguanta 100 usuaris i un que n'aguanta milions.

La ciberseguretat no és només saber detectar atacs és dissenyar sistemes que per la seva estructura fan molt difícil que els accessos no autoritzats passin inadvertits. El nostre simulador IAM és un primer pas sòlid en aquesta direcció.